QUE · TECHNICAL DOCUMENTATION

System Design, Architecture & Testing Guide

Product: **Que** (schema relationship workspace)
Codebase: `adc/schemagraph`
Document version: 1.0 · Generated: 24 Jul 2026
Audience: engineers, QA, architects

Que is an AI-augmented data-engineering workspace for **cross-source schema stitching**. It visualizes tables/collections and join relationships, answers questions from **schema metadata only** (no raw warehouse row centralization), and turns chat drafts into reviewable exportable jobs — all behind workspace ACL (owner / admin / member / viewer).

Table of Contents

1. Executive overview

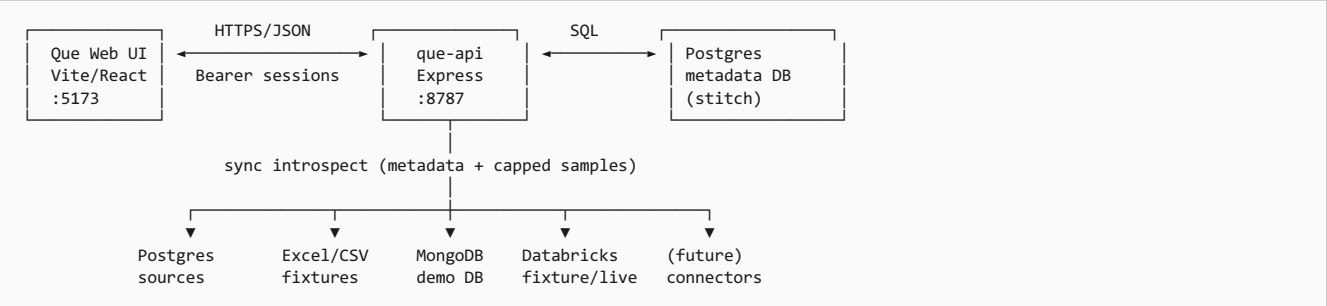
Que helps data engineers understand how schemas relate across heterogeneous sources (PostgreSQL, Excel/CSV, MongoDB, Databricks, etc.) without moving raw customer data into a central lake for AI.

- **Visualize** — interactive diagram of tables, columns, and relationships
- **Infer** — suggest cross-source joins from metadata heuristics
- **Ask** — schema-only AI chat (heuristic or optional LLM)
- **Ship** — draft → ready → export jobs (JSON/SQL artifacts)
- **Govern** — workspace membership + role-gated writes

> Privacy wedge: connectors may capture *capped* column samples for typing/UX, but Que does not centralize full warehouse tables for model training or chat prompts. Chat prompts use schema context packs only.

2. System design & architecture

2.1 High-level components



2.2 Architectural principles

1. **Metadata DB is source of truth for the diagram** — not the customer warehouse.
2. **Connectors are pull/introspect** — sync upserts objects/columns/relationships into Que metadata.
3. **Canvas layout is UX state** — positions live in `diagram_layouts`, separate from schema truth.
4. **AI sees packs, not dumps** — `schemaContext` builds a bounded pack for chat.
5. **ACL is workspace-scoped** — every `/workspaces/:id/*` route checks membership + role rank.

2.3 Request path (authenticated)

1. UI stores Bearer token in `localStorage`.
2. `apiFetch` attaches `Authorization`; on HTTP 401 dispatches auth-expired event → logout.
3. API resolves session hash → user; then membership → `req.workspaceRole`.
4. Mutating routes call `requireMinRole('member'|'admin')`.

3. Technology stack

Layer	Technology	Notes
Frontend	React 19, TypeScript, Vite 8, React Router 7, Tailwind v4, styled-components	Default port 5173
API	Node ESM, Express 4, `pg`	Default port 8787
Metadata DB	PostgreSQL (Docker `stitch-pg`)	DB/user/pass: `stitch`
Demo sources	Postgres `customer_demo`, Mongo `stitch-mongo`	Optional bootstraps
LLM (optional)	OpenAI / Anthropic env keys	Default chat = heuristics

There is **no application Dockerfile** for Que itself; Postgres/Mongo are run as named containers.

4. Repository layout

```
adc/schemagraph/
├── src/                # React SPA (Que UI)
│   ├── pages/          # Login, Workspace, Chat, Jobs, Sources, Settings
│   ├── components/     # Canvas, TopBar, sidebars, MobileNav
│   ├── layouts/        # MainDiagramLayout, QueAppChrome
│   ├── context/        # Auth, Diagram, Toast
│   ├── services/       # auth, stitchApi, apiConfig
│   ├── hooks/          # useWorkspaceRole, useTableNodeDrag
│   └── data/           # Dummy schema/sources (offline fallback)
├── api/
│   ├── src/            # Express app, auth, sync, chat, jobs, connectors/
│   └── fixtures/       # CSV/XLS + Databricks fixture JSON
└── db/                 # SQL migrations 001–005
```

5. Data model (metadata Postgres)

Migrations: `db/001_init.sql` … `db/005_auth.sql`.

Table	Purpose
`users`	Accounts (+ `password_hash`)
`workspaces`	Tenant silo (+ `settings_json`)
`workspace_members`	user ↔ workspace role
`sessions`	Bearer token hashes + expiry
`connections`	Data sources (config JSON, status)
`schema_objects`	Tables / collections / views
`schema_columns`	Columns + key kinds + optional samples
`relationships`	Explicit / AI-inferred edges + review status
`diagram_layouts`	Canvas node positions (JSONB)
`schema_snapshots`	Point-in-time packs after sync
`jobs`	Draft → ready → exported artifacts

6. Authentication & authorization

6.1 Session model

- Login returns random Bearer token; API stores **SHA-256(token)** in `sessions`.
- Default TTL: 14 days (`STITCH_SESSION_DAYS`).
- Passwords: Node `scrypt` (`salt:hash`).
- FE keys: `stitch_auth_token`, `stitch_auth_user`, `stitch_workspace_id`.
- 401 → `notifyAuthExpired()` clears storage and AuthContext (no silent dummy schema).

6.2 Role matrix

Capability	viewer	member	admin	owner
Read schema/sources/jobs/settings/context	Yes	Yes	Yes	Yes
Layout, promote/reject, sync, chat, jobs write/export	No	Yes	Yes	Yes
Connection CRUD, settings PATCH	No	No	Yes	Yes

6.3 Demo accounts (seeded at API boot)

Email	Password	Role
`dev@stitch.local`	`stitch-dev`	owner
`member@stitch.local`	`stitch-member`	member
`viewer@stitch.local`	`stitch-viewer`	viewer

Demo workspace ID: `22222222-2222-2222-2222-222222222222` (slug `demo`).

> `STITCH_AUTH_DISABLED=true` bypasses checks and injects the owner user — local prototyping only.

7. Frontend application design

7.1 Routes

Path	Page	Auth
`/login`	LoginPage	Public
`/workspace`	WorkspacePage + MainDiagramLayout	Required
`/chat`	ChatPage	Required
`/sources`	SourcesPage	Required
`/jobs`	JobsPage	Required
`/settings`	SettingsPage	Required

7.2 Workspace layout regions

- **TopBar** — brand Que, nav / mobile Menu, search, tools (filters/export), session
- **Left sidebar** — connections; click filters canvas by `sourceId`; Sync Schema (member+)
- **MainCanvas** — pan/zoom, table nodes, SVG relationships, minimap
- **Right sidebar** — selected table details

7.3 Offline / error behavior

- **401 / 403** on load → blocked screen (no dummy graph).
- **Network / other errors** → dummy schema + orange “API offline” banner.
- Viewers: canvas `readOnly` (no drag persist); mutate actions hidden/disabled.

8. API reference

Base URL: `http://localhost:8787` · Service health: `GET /health` → `{ ok, service: 'que-api', authDisabled }`

Method	Path	Min role	Purpose
POST	`/auth/login`	—	Issue session
POST	`/auth/logout`	—	Revoke session
GET	`/auth/me`	auth	User + workspaces
GET	`/workspaces`	auth	List workspaces
GET	`/workspaces/:id/schema`	viewer+	Graph payload
GET	`/workspaces/:id/sources`	viewer+	Connections
POST/PATCH/DELETE	`.../connections`	admin+	CRUD sources
POST	`.../connections/:cid/sync`	member+	Introspect
PUT	`.../layout`	member+	Save positions
PATCH	`.../relationships/:rid`	member+	promote/reject
POST	`.../chat`	member+	Schema chat
GET/PATCH	`.../settings`	viewer+ / admin+	Policies
GET/POST/PATCH	`.../jobs`	viewer+ / member+	Job lifecycle
POST	`.../jobs/:jid/export`	member+	JSON or SQL artifact

9. Connectors & sync pipeline

Type	Module	Behavior
postgresql	`connectors/postgres.js`	information_schema + keys; capped samples
excel / csv	`connectors/spreadsheet.js`	Files under `api/fixtures/`
mongodb	`connectors/mongo.js`	Collection schema from sampled docs
databricks	`connectors/databricks.js`	Fixture JSON or live SQL Statement API

Sync steps

1. Load connection + workspace prefs (`includeSamplesDefault`).
2. Introspect via connector.
3. Upsert `schema_objects` / `schema_columns` ; prune removed.
4. Replace explicit FK relationships for synced objects.
5. Mark connection active; write `schema_snapshots` .
6. Optionally `inferCrossSourceJoins` → `ai-inferred` + `suggested` edges.

10. Schema graph & promote/reject

- `relation_type` : `explicit` | `ai-inferred`
- `status` : `suggested` | `accepted` | `rejected`
- GET schema excludes `rejected` .
- **Promote** → accepted + explicit.
- **Reject** → rejected (removed from canvas).

Smoke-test AI edge ID: `ddddddd-dddd-dddd-dddd-ddddddddddd3` (`analytics_cube.user_id → users_main.id`).

11. Chat → Jobs → Export flow

```
User (/chat)
   │  POST /chat  { message, history }
   ▼
buildSchemaContextPack() ──► heuristic | LLM (if preferLlmChat + key)
   │
   ▼
{ reply, citations, jobDraft?, sql?, referencedTables, mode }
   │  Save draft (member+)
   ▼
POST /jobs  (status: draft)
   │  JobsPage: Mark ready
   ▼
PATCH /jobs/:id  (status: ready)
   │  Export
   ▼
POST /jobs/:id/export  { format: json|sql }
   → status: exported + downloadable artifact
   (no warehouse execution yet)
```

Job statuses: `draft` → `ready` → `exported` | `archived`.

12. Workspace settings & policies

Flag	Default	Effect
`includeSamplesDefault`	true	Capped samples on sync
`inferJoinsOnSync`	true	Cross-source join suggestions
`preferLlmChat`	false	Prefer LLM when keys exist

PATCH settings requires **admin+**. GET also returns stats, latest snapshot, connector/LLM capability flags, brand `Que`.

13. End-to-end user flows

13.1 Owner — first session

1. Open `http://localhost:5173/login`.
2. Sign in as owner demo account.
3. Land on `/workspace`; schema loads from API.
4. Use Tools filters / search; drag tables (layout saves).
5. Hover AI edge → Promote or Reject.
6. Sidebar → Sync Schema on a syncable connection.
7. Chat → ask "list tables" → optionally Save job → Jobs → Mark ready → Export SQL/JSON.
8. Settings → toggle policies → Save.

13.2 Viewer — read-only

1. Login as viewer.
2. Can view workspace/sources/jobs/settings.
3. Cannot sync, chat, promote, save settings, or CRUD connections.
4. Canvas shows VIEW ONLY; drag does not persist.

13.3 Member — write without admin

1. Login as member.
2. Can sync, chat, jobs, layout, promote/reject.
3. Cannot create/edit/delete connections or patch settings (403).

14. Local runbook (Windows PowerShell)

14.1 First-time

```
# Metadata Postgres
docker run -d --name stitch-pg `
  -e POSTGRES_USER=stitch -e POSTGRES_PASSWORD=stitch `
  -e POSTGRES_DB=stitch -p 5432:5432 postgres:16

Get-Content d:\ADC\prosols\adc\schemagraph\db\001_init.sql |
  docker exec -i stitch-pg psql -U stitch -d stitch
Get-Content d:\ADC\prosols\adc\schemagraph\db\003_jobs.sql |
  docker exec -i stitch-pg psql -U stitch -d stitch
Get-Content d:\ADC\prosols\adc\schemagraph\db\004_workspace_settings.sql |
  docker exec -i stitch-pg psql -U stitch -d stitch
Get-Content d:\ADC\prosols\adc\schemagraph\db\005_auth.sql |
  docker exec -i stitch-pg psql -U stitch -d stitch

# Optional demo sources
docker run -d --name stitch-mongo -p 27017:27017 mongo:7
cd d:\ADC\prosols\adc\schemagraph\api
npm install
npm run bootstrap:demo-source
npm run bootstrap:demo-mongo
npm run seed
```

14.2 Daily

```
docker start stitch-pg
# Terminal A
cd d:\ADC\prosols\adc\schemagraph\api; npm start
# Terminal B
cd d:\ADC\prosols\adc\schemagraph; npm run dev
```

UI: `http://localhost:5173` · API: `http://localhost:8787`

15. Testing strategy & test flows

> The repo currently has no automated Jest/Vitest/Playwright suite. Validation is manual + API smoke scripts. FE lint: `npm run lint` (oxlint). Build: `npm run build`.

15.1 Smoke checklist — API

1. `GET /health` → ok, `authDisabled: false`.
2. Login owner / member / viewer → correct roles.
3. No token on `/schema` → 401.
4. Viewer: schema 200; sync/settings/chat/layout/create → 403.
5. Member: layout 200; settings PATCH → 403.
6. Owner: settings PATCH 200; chat 200.
7. Logout → `/auth/me` 401.
8. Garbage Bearer → 401.

15.2 Smoke checklist — UI

1. Login form responsive; demo account buttons fill credentials.
2. Workspace loads live schema (not dummy) when API up.
3. Mobile Menu reaches Chat/Sources/Jobs/Settings.
4. Tools panel: filters + JSON/PNG/PDF export.
5. Source click filters tables; Sync shows toast.
6. Promote/reject inferred edge; toast feedback.
7. Viewer: VIEW ONLY, no Sources ADD, chat disabled.
8. Stop API → offline banner + dummy data (not auth error).
9. Revoke session / expire token → redirected to login (not dummy).

15.3 Role matrix regression

Action	viewer	member	owner
View diagram	Pass	Pass	Pass
Drag + persist layout	Fail/blocked	Pass	Pass
Sync connection	403	Pass	Pass
Create connection	403	403	Pass
Chat send	403	Pass	Pass
Settings save	403	403	Pass

15.4 Suggested future automation

- API contract tests (supertest) for auth/ACL matrix.
- Playwright: login → workspace load → promote → sync → chat → export.
- Visual regression on TopBar / Login (Midnight Cyber).

16. Security & privacy notes

- Secrets should live in connection config / env — FE sources API redacts where implemented.
- Sessions are hashed at rest; transport must be HTTPS in production.
- Chat system prompt asserts schema-only; still validate no raw dumps in packs.
- Disable `STITCH_AUTH_DISABLED` outside local machines.
- Rotate demo passwords before any shared deployment.

17. Known limitations & roadmap notes

- Job “export” produces artifacts; does not execute SQL on warehouses.
- Databricks live mode needs real cloud credentials; fixture mode is the default demo.
- Some internal identifiers still use `stitch_*` prefixes (env, localStorage, docker names).
- PDF export uses browser print of a summary document; PNG is SVG-rendered snapshot.
- No multi-region HA / SSO / SCIM in this build.

18. Appendix — key files & demo IDs

Area	Path
API routes	`api/src/index.js`
Auth / RBAC	`api/src/auth.js`, `src/services/auth.ts`, `src/hooks/useWorkspaceRole.ts`
Sync	`api/src/syncConnection.js`, `api/src/connectors/*`
Chat	`api/src/chatEngine.js`, `api/src/schemaContext.js`
Jobs	`api/src/jobs.js`
Canvas	`src/pages/WorkspacePage.tsx`, `src/components/MainCanvas.tsx`
FE API client	`src/services/stitchApi.ts`, `src/services/apiConfig.ts`

Entity	ID
Demo workspace	`22222222-2222-2222-2222-222222222222`
Owner user	`11111111-1111-1111-1111-111111111111`
pg_customer_demo	`aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaa4`
AI smoke edge	`dddddddd-dddd-dddd-dddd-ddddddddddd3`

End of document — Que Technical System Documentation v1.0
Source of truth: codebase under `adc/schemagraph`. Update this PDF when auth, connectors, or job execution change.